02. Tests unitaires SQL

Principe général

Les exemples de tests unitaires fournis dans le projet de base ont une structure qui ressemble à ceci :

```
begin;
do
$test$
    begin
        raise notice 'TEST: exemple de test unitaire';

        /*
        Contenu du test:

        Par exemple plusieurs insert/update qui permettent de vérifier qu'une
règle métier spécifique
        est correctement implémentée. Ces commandes vont vérifier que des cas
valides sont effectivement
        autorisés et/ou que des cas invalides sont refusés et déclenchent une
exception.
        */
    end
$test$;
rollback;
```

Le code compris entre le premier `begin` et le `rollback` constitue un bloc de code que l'on veut *transactionnel*, c'est-à-dire dont l'exécution se fait en mode "tout ou rien" : tout est sauvé en base de données ou rien du tout. Une telle transaction se termine généralement par un `commit` qui permet de signaler la fin de la transaction de manière positive, confirmant la persistence en BD de toutes les opérations réalisées.

Dans le cas spécifique des tests unitaires, on remplace `commit` par `rollback` qui a l'effet contraire : il annule toutes les opérations réalisées durant la transaction. L'avantage de cette approche est que les modifications effectuées durant la transaction pour vérifier que la règle métier est bien vérifiée sont supprimées en fin de transaction, laissant la base de données dans le même état qu'avant l'exécution du test. Ceci permet d'avoir des tests unitaires idempotents.

Cas positifs

Voyons comment implémenter un test unitaire qui traite un cas "positif", c'est-à-dire une modification qui doit être autorisée.

Pour cela, on va modifier l'email de l'utilisateur 1 en `boris.verhaegen@epfc.eu`, ce qui est une opération légale puisque cette adresse mail n'est utilisée par personne d'autre.

```
begin;
do
$test$
    begin
        raise notice 'TEST: email du user est unique (cas valide)';
        update users set email = 'boris.verhaegen@epfc.eu' where id = 1;
    end
$test$;
rollback;
```

L'exécution de ce bloc de code donne le résultat suivant :

```
TEST: email du user est unique (cas valide)
[2026-03-30 13:19:15] completed in 2 ms
[2026-03-30 13:19:15] prbd-2526-xyy.public> rollback
[2026-03-30 13:19:15] completed in 1 ms
```

Le test est valide et aucune erreur n'est détectée. En outre, grâce au `rollback`, vous pouvez vérifier que l'adresse email de l'utilisateur 1 n'a pas été modifiée et possède toujours sa valeur initiale (`boverhaegen@epfc.eu`).

Cas négatifs

Voyons comment implémenter un test unitaire qui traite un cas "négatif", c'est-à-dire une modification qui doit être refusée.

Pour cela, on va modifier l'email de l'utilisateur 1 en `bepenelle@epfc.eu`, ce qui est une opération illégale puisque cette adresse mail est déjà utilisée et que nous avons une contrainte d'unicité sur la colonne `email`.

```
begin;
do
$test$
    begin
        raise notice 'TEST: email du user est unique (cas invalide)';
        perform should_fail($$
            update users set email = 'bepenelle@epfc.eu' where id = 1;
        $$, 'unique_violation');
    end
$test$;
rollback;
```

L'exécution de ce bloc de code donne le résultat suivant :

```
TEST: email du user est unique (cas invalide)
[2026-03-30 13:25:58] completed in 2 ms
```

```
[2026-03-30 13:25:58] prbd-2526-xyy.public> rollback
[2026-03-30 13:25:58] completed in 2 ms
```

Le test est valide et aucune erreur n'est détectée. En effet, la fonction `should_fail` a exécuté l'update et a constaté que l'exception `unique_violation` a bien été déclenchée.

Exception non capturée dans un cas de test

Imaginons que vous êtes en train d'écrire et de tester un test unitaire et que votre code déclenche une exception que vous ne capturez pas. Par exemple, vous faites un update que vous pensez valide, mais qui s'avère ne pas l'être ou inversément.

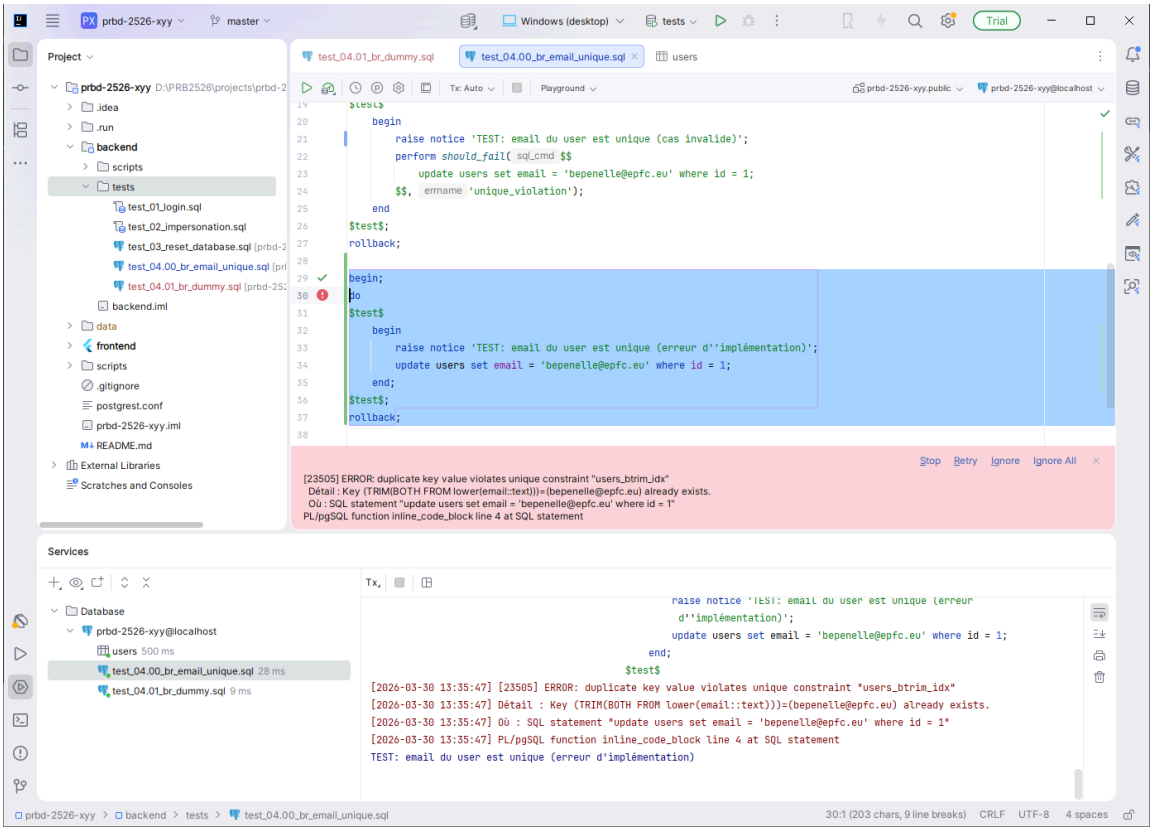
Par exemple, le test suivant déclenche une exception car l'update n'est pas valide. Cette exception devrait être capturée par un `should_fail` :

```
begin;
do
$test$
    begin
        raise notice 'TEST: email du user est unique (erreur
d''implémentation)';
        update users set email = 'bepenelle@epfc.eu' where id = 1;
    end;
$test$;
rollback;
```

Dans cet autre exemple, le test déclenche une exception, car l'update ne déclenche pas d'exception alors que le `should_fail` attend (à tort) qu'il en déclenche une.

```
begin;
do
$test$
    begin
        raise notice 'TEST: email du user est unique (cas invalide)';
        perform should_fail($$
            update users set email = 'boris.verhaegen@epfc.eu' where id = 1;
        $$, 'unique_violation');
    end
$test$;
rollback;
```

Dans ces cas-là, l'exécution de la transaction est interrompue par IntelliJ qui affiche la ligne ayant provoqué l'erreur :



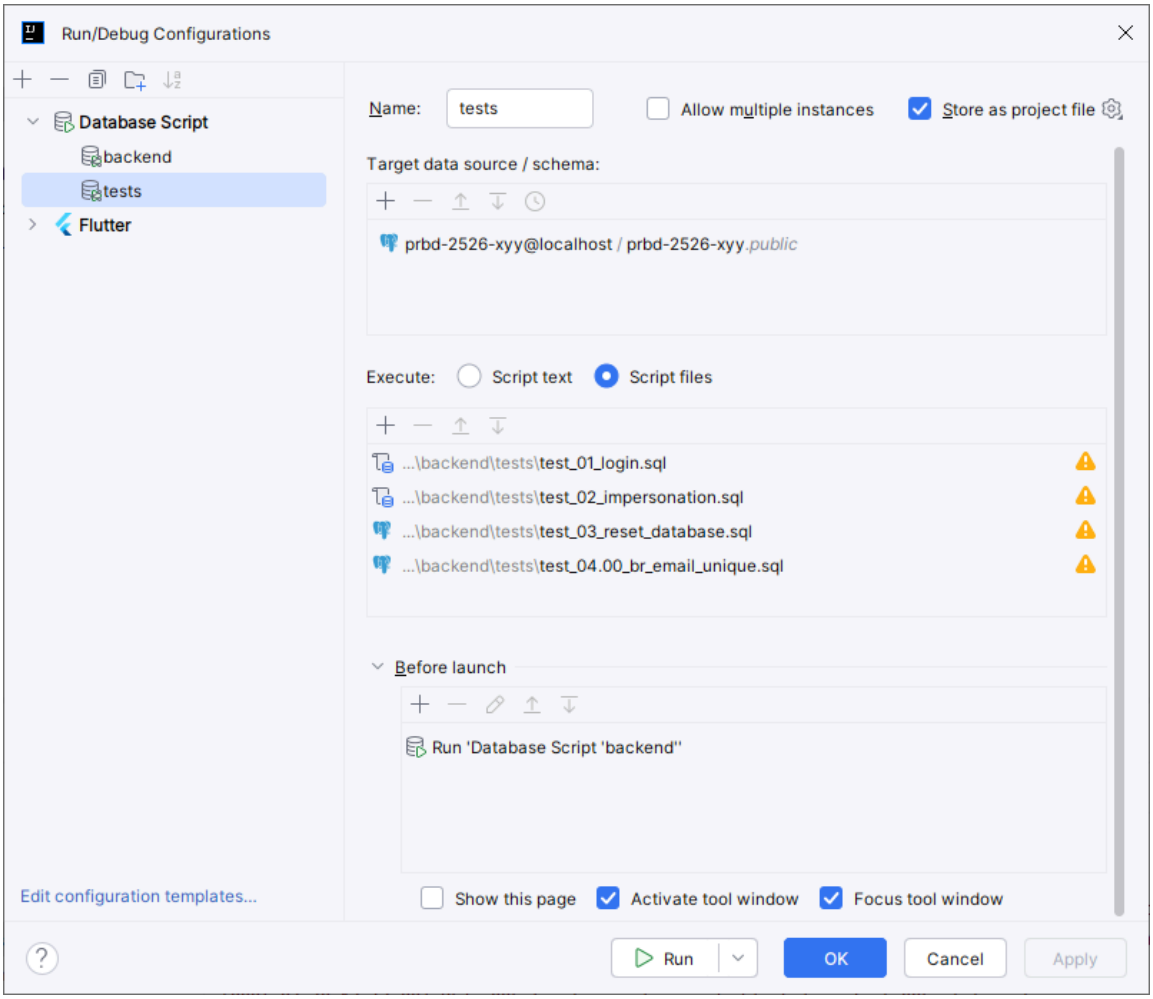
Il est important de comprendre que la transaction a été interrompue et est toujours en cours. Si vous cliquez sur "Stop" et que vous réexécutez le bloc ou si vous cliquez sur "Retry", vous obtenez le message suivant "ERROR: current transaction is aborted, commands ignored until end of transaction block" qui vous indique clairement que la transaction est en cours et que toutes les commandes que vous pouvez exécuter dans ce contexte seront ignorées.

Pour sortir de ce mode et finaliser la transaction, cliquez sur "Ignore All" ou bien exécutez la commande `rollback`.

Vous pourrez ensuite corriger le code de votre test unitaire, ou celui de votre règle métier le cas échéant, pour qu'il n'y ait plus d'exception déclenchée lors de l'exécution du test unitaire.

Automatisation de l'exécution des tests unitaires

Pour pouvoir exécuter en batch tous vos tests unitaires, vous pouvez regrouper tous leurs fichiers dans la configuration d'exécution "tests" du projet dans IntelliJ (même principe que pour l'exécution du backend) :



Vous remarquerez aussi que cette configuration est paramétrée pour exécuter la configuration "backend" préalablement à sa propre exécution (voir "Before launch" dans la capture d'écran). Ceci permet de garantir le fait que les tests unitaires s'exécutent sur une base de données pristine.

Cas des triggers différés

Avec un `rollback` final, les déclencheurs **différés** ne sont pas vérifiés puisqu'ils sont censés être vérifiés au `commit` de la transaction qui n'a pas lieu. Dès lors, un scénario invalide pourrait alors faire **passer** le test à tort.

Pour éviter cela, on doit terminer le bloc de commandes par `SET CONSTRAINTS ALL IMMEDIATE` après les dernières modifications. C'est déjà ce que fait la fonction `should_fail`. En revanche, il est nécessaire de le faire manuellement dans le cas des tests unitaires positifs. Voir la FAQ 07. Faux positifs dans les tests unitaires.